LastBite

Good food. Second chance.

01 DevSecOps capstone | Python, React and PostgreSQL

02 Surplus meal listings, stock reservations and code-verified pickup

The application

Restaurant surplus becomes someone's next meal

01 Publish a meal box with stock, prices, allergens and a pickup window.

02 Reserve available boxes; prevent overselling with atomic stock updates.

03 Cancel to release stock, or confirm collection using the pickup code.

04 Only collected boxes count toward meals rescued and savings.

Technology stack

A complete application and delivery platform

01 Interface: React and Vite, served by unprivileged Nginx.

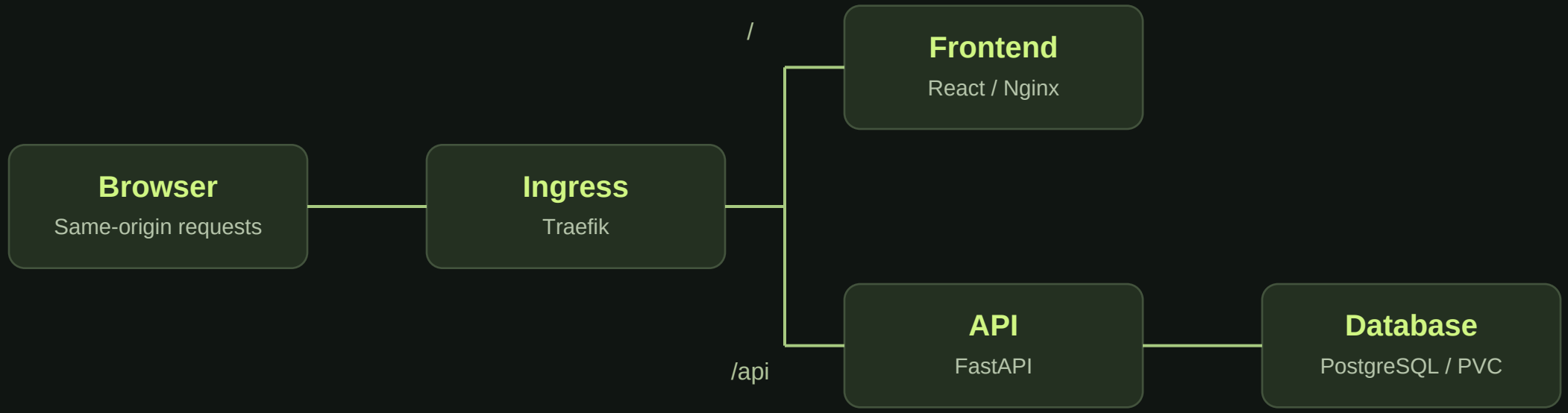
02 API and data: FastAPI, SQLAlchemy, PostgreSQL and Alembic.

03 Delivery: GitHub Actions, Docker, Trivy, GHCR and Argo CD.

04 Operations: Terraform, AWS EKS, Helm, Prometheus and Grafana.

Runtime architecture

One browser origin; separate application services



Prometheus scrapes the API. Grafana visualizes traffic, errors and latency.

CI and release promotion

Only scanned images reach the registry

01 A push or pull request runs API tests, a UI build and infrastructure checks.

02 CI builds both images, then scans each for HIGH and CRITICAL findings.

03 A successful main build pushes those exact images using the source SHA.

04 A reviewed release-values commit tells Argo CD which images to deploy.

AWS infrastructure

Terraform defines the network and cluster

01 Two public and two private subnets span two availability zones.

02 Private EKS worker nodes use a NAT gateway for outbound access.

03 The API permits only configured administrator CIDRs.

04 The EBS CSI driver provides encrypted gp3 persistent storage.

Kubernetes and GitOps

Database readiness precedes application traffic

01 Argo CD first creates PostgreSQL, then runs an Alembic migration job.

02 Backend readiness requires the migrated database; liveness stays cheap.

03 Two frontend replicas and a backend HPA support the deployment demo.

04 Passwords come from a Kubernetes Secret created outside Git.

Observability

Metrics make failures and load visible

01 Dashboard panels show request rate, P95 latency, errors and replicas.

02 ServiceMonitor labels connect the backend to Prometheus discovery.

03 Alert rules detect sustained errors and missing scrape targets.

04 The load script creates API traffic; CPU scaling depends on actual load.

Engineering lessons

The details between tools determine reliability

01 A security scan needs an explicit nonzero exit code to stop promotion.

02 Service names, ingress ports and Nginx upstreams must agree.

03 Schema migration belongs in one job, not every application replica.

04 Concurrent API tests verify stock, pickup rules and migration behavior.

Live demonstration

Evidence to capture in the final recording

01 Publish, reserve, cancel and collect a meal box; show updated impact.

02 Show a real CI run, image scans, SHA tags and Argo CD sync.

03 Show Terraform output, healthy pods, Prometheus targets and Grafana.

04 Explain the isolated failure lab, then record infrastructure cleanup.